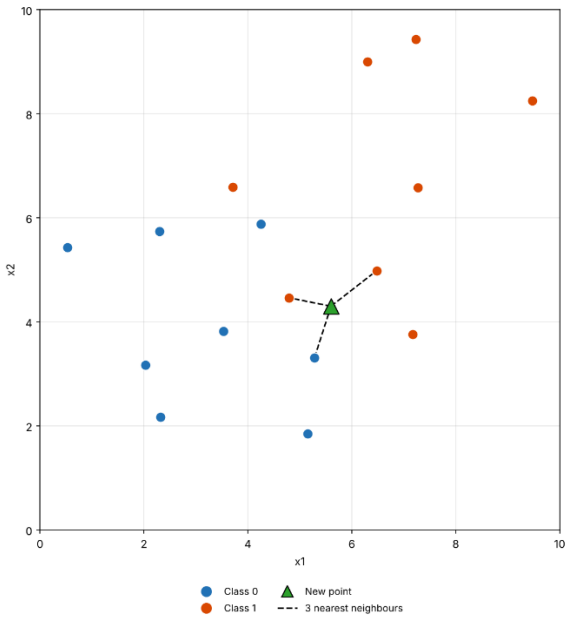
# Introduction to Machine Learning

Session 5 – Lecture Notes

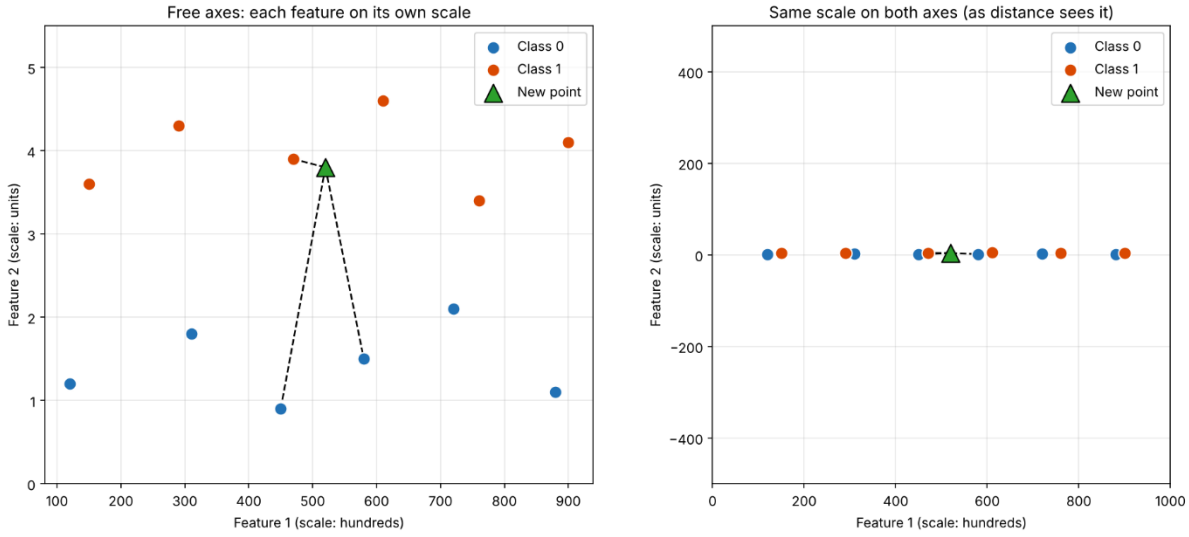
## Non–Parametric Models: K–Nearest Neighbours

It classifies by distance (most often Euclidean distance – $d = \sqrt{\Delta x_1^2 + \Delta x_2^2}$ )




K =3, Δ = Class 0.

The different scaling of the features is a problem.



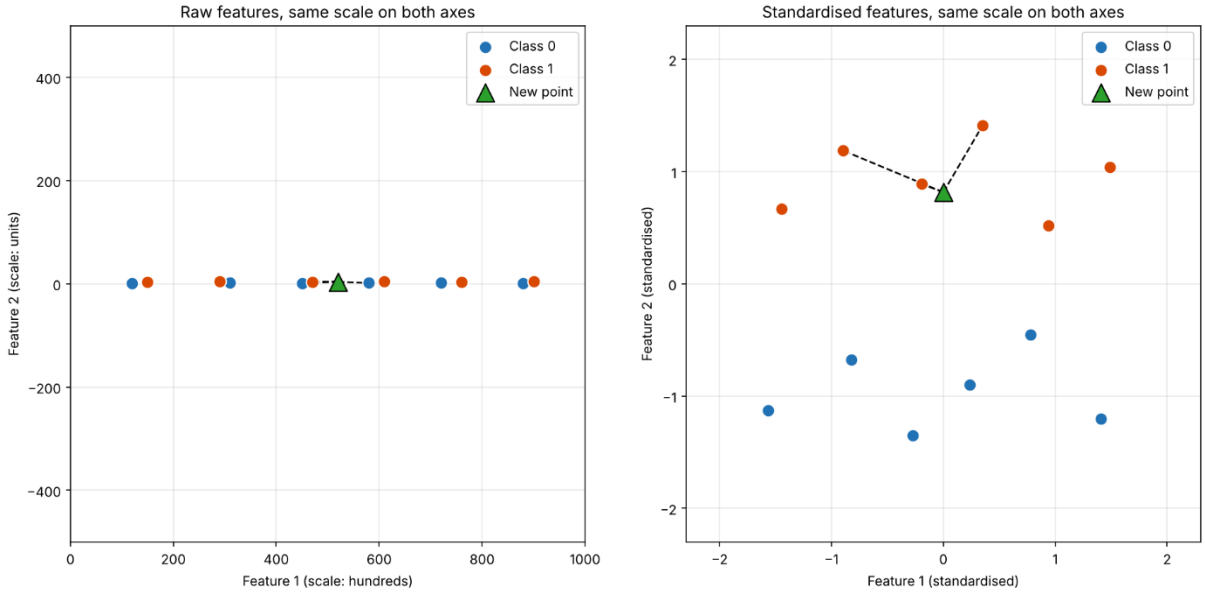

K =3, Δ = Class 0.

When utilizing Euclidean distance, feature imbalance is a big problem – the influence of each variable is different.

## Z–Score Standardization

$$x_1' = \frac{x_1 - \mu_1}{\sigma_1}$$

To remove imbalance and give each feature same influence.




K =3, Δ = Class 1.

*The model carries information about k, μ & σ per feature, and standardized train data with class assignment.*

*The μ & σ are computed on train set only.*

## Distance Metrics

The Minkowski Family of Distances

- p = 1 (Manhattan) – sum of absolute distances (city block)
  $d = |\Delta x_1| + |\Delta x_2|$
- p = 2 (Euclidean)
  $d = \sqrt{\Delta x_1^{\,2} + \Delta x_2^{\,2}}$
- p → ∞ (Chebyshev) – only the single largest feature difference counts
  $d = max_{i=1}^{n} ( |\Delta x_i| )$

Mahalanobis distance – takes into consideration both the means and distributions of features

$$d = \sqrt{(\Delta x^T C^{-1} \Delta x)}$$

Others

- Cosine – ignores magnitude & compares direction only
- Hamming – counts how many features differ. (Good for categorical data)
- Jaccard – for sets. Which tags two scenes share
- Gower – for mixed numerical/categorical data

- num → absolute distance divided by the range
- cat → 1 or 0

**Drawbacks of KNNs:**

The Curse of Dimensionality As you grow in features (dimensions), the volume of the space grows so fast that data becomes very sparse.

The dataset is carried through with the algorithm → so it can be quite heavy.

It's either susceptible to noise or to generalization.

*KNN is simple, the representation is the hard part.*

## Classification Trees

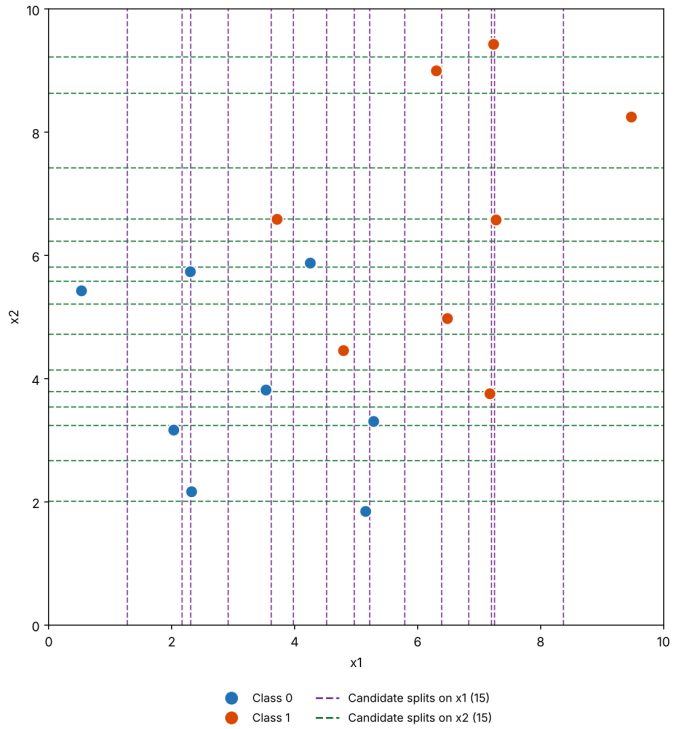
A series of Yes/No questions, selected greedily.

*Gini impurity:* $G = 1 - \Sigma_k p_k^2$

*They both prioritize the purity of the split.*

*Entropy:* $H = -\Sigma p_k \cdot \log_2(p_k)$

### Brute–Force Search




Then the next split works only for the subsets of the data, until we reach 100% accuracy.

The risk for overfitting is very real, and a small input (training data) change is equal to a big model change!

## Random Forest

The tree is extremely dependent on the sample → different sample = different model.

So, they are very susceptible to random errors.

- We are thus bagging (bootstrap aggregating): n points drawn with.

*n = n training points.* *≈ 63% unique points.*

- We select features at random too… (and at every branch!)

*m = √k, where k I s number of features, rounded down to at least 1*

So that no feature remains dominant.

Then:

- *Soft vote → average of probabilities*
- *Hard vote → dominant class*

Difference if we stop the trees growing on the same layer.

You can also get probabilities as output, either from the average, or from n/m (n = minority–class trees, m = all trees) – which is informative in its own way.

Compute and memory grow with the scale.

About 100–500 trees in typical forest.